BeaconMind: Can a Large Language Model Own Landmark SLAM?

Xiaohui Chen
Engineering for Professionals, Whiting School of Engineering
Johns Hopkins University
Baltimore, United States
EN.665.715 Embodied Agentic AI for Robotic Control (individual project)

***Abstract*—Reserved for a later draft. This submission covers only Section II (Related Works) and the References; the other sections are placeholders.**
***Index Terms*—Reserved for a later draft.**

I. Introduction

Reserved for a later draft.

II. Related Works

BeaconMind, the system proposed in this project, asks which parts of landmark-based simultaneous localization and mapping (SLAM) a large language model (LLM) can own, that is, for which parts the model itself can make the estimation decisions. A low-cost differential-drive car with wheel encoders, an inertial measurement unit, and one monocular camera, but no range sensor, explores a walled arena whose only landmarks are AprilTags printed on paper and fixed to the walls. Each physical tag is a *beacon*; because tag identifiers may repeat in the benchmark, an identifier does not always name one beacon. The car moves in a stop-observe-decide-move cycle. At each stop, an LLM agent receives the odometry increment and the tag detections as text, obtains every geometric quantity from deterministic tools, and maintains its estimate of the car's pose (the pose belief) and the beacon map: it decides which detections to trust, which beacon a detection belongs to, whether a pose fix or a loop closure (a recognized revisit) is accepted, and which of a few fixed, short moves to make next. The project is at the design stage, so statements about BeaconMind describe the planned system and its planned evaluation, not results. Four strands of prior work shape the design; for each, this section summarizes what was shown, what it leaves open, and how BeaconMind differs.

A. Language-Conditioned Planning and Closed-Loop Agents

A first line of work made LLM plans executable by grounding them in what the robot can do. Huang *et al.* showed that a pretrained LLM decomposes a household task into plausible steps, but that naively generated plans often cannot be mapped to admissible actions [1]. SayCan addressed this by weighting each candidate skill with a learned affordance value; removing that grounding lowered planning success from 84% to 67% over 101 instructions [2]. Inner Monologue then returned success detection and scene descriptions to the planner as text and found that closed-loop language feedback significantly improves instruction completion [3]. ReAct formulated the pattern in general form, interleaving reasoning, action, and observation, and outperformed imitation and reinforcement learning baselines on the ALFWorld and WebShop benchmarks by 34 and 10 percentage points in success rate, respectively [4]. Kambhampati *et al.* argue that this line of work has a limit: LLMs can neither plan nor verify plans by themselves and, in what the authors call an LLM-Modulo framework, should generate candidates that external, sound verifiers check [5].

These ideas are now packaged for robot software. ROSA wraps the Robot Operating System (ROS) in a ReAct agent with predefined tools so that operators can command robots in natural language [6], and the survey by Salimpour *et al.* classifies such integrations from single-shot protocol translation, through ReAct-style agentic interfaces and orchestration, to end-to-end embedded policies [7]. Benchmarks quantify what still fails. The Embodied Agent Interface reports that trajectory-feasibility errors are common (45.2%), a large portion of them missing-step and additional-step errors [8], and in SafeAgentBench the most safety-conscious agent rejects only 10% of detailed hazardous tasks [9].

BeaconMind adopts these lessons. Its agent is a ReAct-style agentic interface in the survey's classification, and ROSA is its planned demonstration runtime; the benchmark will instead use a minimal harness that exposes only the project's own tools, because stock ROSA adds its own tools and fixed prompts to every call and would confound a comparison of models. Its actions are admissible by construction, because the agent may choose only from a small fixed set of short moves, and its loop is closed, because every decision starts from a fresh observation. The missing-step and additional-step errors that the Embodied Agent Interface reports cannot arise in that form, since the agent emits one move per observation instead of a multi-step plan. Following the LLM-Modulo argument, a deterministic validator between the model and the motors rejects any move outside that set or beyond the safety limits. Because the agent also accepts free-text operator requests, its refusal of unsafe ones is proposed to be measured with a fixed request suite and a rejection rate adapted from SafeAgentBench. The difference lies in what the model reasons about: the works above plan *tasks* and take the robot's state as given, whereas BeaconMind asks the model to estimate that state,

TABLE I
DIVISION OF LABOR BETWEEN THE LANGUAGE MODEL AND CLASSICAL GEOMETRY IN REPRESENTATIVE PRIOR SYSTEMS AND IN BEACONMIND

System	Pose and map supplied by	What the language model contributes
SayPlan [10]	Pre-constructed 3D scene graph	Task planning over the scene graph
VLFM [11]	Odometry sensor; occupancy map built from depth	Scores which frontier to explore (vision-language model)
L3MVN [12]	Odometry sensor; semantic map from RGB-D images	Selects the frontier relevant to the target object
Lee *et al.* [13]	Cartographer SLAM (Nav2 executes the goals)	Picks goals from annotated map images; logs semantics
Syarubany *et al.* [14]	ORB-SLAM3 global pose; AprilTags as anchors	One constrained discrete command at each junction
SEO-SLAM [15]	Odometry, linear-assignment association, factor graph	Object verification, label refinement, and landmark merging
Sandfuchs *et al.* [16]	Oracle localization in a text gridworld	One grid move per step, to explore or to navigate
BeaconMind (this project, planned)	The LLM's pose belief and beacon map, updated at every stop with single-timestep geometry tools; in the primary design a classical factor graph serves only as the reference	Which detections to trust, data association under duplicated tag identifiers, acceptance of pose fixes and loop closures, map updates, and the choice of each move

the pose and the map that such plans presuppose.

B. Language Models in Navigation, Mapping, and SLAM

In the language-driven navigation systems reviewed here, the geometry stays classical (Table I). SayPlan plans over a pre-constructed 3D scene graph and delegates path planning to Dijkstra's algorithm; its authors report that the LLM fails at simple distance-based reasoning and name the pre-built, static graph as a limitation [10], whereas BeaconMind's map does not exist before the run and is built by the agent. For unknown spaces, VLFM builds an occupancy map from depth observations and uses a vision-language model to score frontiers (the boundaries between explored and unexplored space) and pick the next one to visit [11], and L3MVN has an LLM select the relevant frontier from a semantic map built from RGB-D images [12]; both report strong object-goal navigation results in simulation and a real-robot demonstration. Both also need depth to build the map and take the robot's position from an odometry sensor; BeaconMind has no range sensor, so its map is a set of beacon coordinates, not an occupancy grid. Lee *et al.* connect LLMs to ROS through Model Context Protocol tools that render the occupancy grid as annotated images; with Cartographer for SLAM and Nav2 for navigation in Isaac Sim, Claude Sonnet 4.6 and GPT-5.5 reach 97.8% and 99.8% map coverage and succeed in 26 and 28 of 30 spatial-reasoning navigation trials, respectively [13]. The closest system identified in the literature search is that of Syarubany *et al.*: an LLM issues one constrained discrete command at each corridor junction, and AprilTags serve as stable anchors, but ORB-SLAM3 provides the global pose and the evaluation of the integrated system is qualitative [14].

Where language models come closer to SLAM itself, they supply semantics or discrete choices, not estimates. SEO-SLAM assigns multimodal LLM agents to object verification, label refinement, and landmark merging, and reports higher semantic accuracy and fewer false positives than its baselines, while data association (deciding which landmark an observation belongs to) is solved as a linear assignment problem over semantic and Mahalanobis distances and the map comes from factor-graph optimization, the standard graph-based formulation of SLAM [15]. Lyu *et al.* let a multimodal LLM make the final choice among place-recognition candidates retrieved by a vision foundation model [17]; this is a language-based form of the place recognition behind loop closure, but its output is a choice among images, not a pose. Sandfuchs *et al.* evaluate nine LLMs as text-only controllers in partially observable gridworlds in which the agent's coordinates are always known, and they state the boundary explicitly: "This is not SLAM: the uncertainty is in the map, not in pose" [16]. The strongest reasoning-tuned models (GPT-5 and GPT-OSS-120B) reach the goal in all three layouts, but with the pose supplied, which is the condition BeaconMind removes. Learned visual navigation policies such as NoMaD sidestep metric estimation altogether: they use no language model, turn camera images directly into actions and, for long horizons, rely on a topological graph of past observations instead of a metric map [18]. BeaconMind lists such a policy as a candidate exploration baseline; it cannot be scored on pose or map error, because it yields no metric pose or map.

In none of the language-model systems above is the robot's pose estimate in the model's hands. SEO-SLAM comes closest on the map side: its agents prune and merge object landmarks produced by a conventional detection, association, and factor-graph pipeline and feed those edits back into the graph. The paper scores the semantic precision and recall of the resulting map and the absolute pose error of the optimized trajectory, but the estimate remains the optimizer's. None of these systems can say how accurate a pose or a geometric map maintained by a language model would be. BeaconMind moves that responsibility to the LLM: deterministic tools still do the arithmetic, but the LLM decides at every stop which results enter the pose belief and the beacon map. In the primary design the classical estimator is kept only as a reference (variants with no tools, or with the LLM on top of the classical estimator, are proposed as stretch conditions), and the LLM's belief is scored against ground truth at every stop. To my knowledge, no prior study reports a trajectory or map error for a landmark-SLAM estimate maintained by an LLM. That measurement is the gap this project targets, and a negative outcome would still be informative.

C. Limits of LLM Spatial Reasoning and the Case for Tools

Recent diagnostics show why handing estimation to a language model is risky, and they shape how BeaconMind does it. Jiang *et al.* find that single-call, whole-route maze navigation

by GPT-4o, DeepSeek-V3, and Llama-3.3-70B falls to near-zero success by 10×10 mazes; that structured coordinate text is the most navigable input format, far ahead of rendered images; and that handing corridor walking and dead-end retreat to a deterministic walker, with the model queried only at junctions through an explicit cell-type prompt, lifts GPT-4o's success by up to 92 percentage points at mid sizes, although the collapse returns by 30×30 [19]. Li *et al.* find that models treat each spatial question independently instead of building cumulative spatial knowledge, and that accuracy depends on the representation: Gemini-2.5-Flash with chain-of-thought prompting reaches 80–86% on small mazes given as adjacency lists but 16–34% when they are given as visual character grids [20]. For perception, Deng *et al.* show that the best vision-language model reaches a macro-F1 of 0.66 on relative camera pose estimation, where a geometric pipeline (LoFTR) reaches 0.99 and humans 0.91 [21]. These results do not contradict Lee *et al.*, whose models read annotated map images to pick goal positions and judge coverage while a classical planner executed every path. On the constructive side, PAL shows that LLMs decompose problems well but make arithmetic mistakes even when the decomposition is correct, and that off-loading the solution step to a program interpreter gives more accurate results than much larger models [22].

BeaconMind is designed around these findings. The model receives the tag detector's output (identifier, range, bearing, and a quality score) as structured text rather than pixels, with raw-image input kept as a separate planned ablation, and it keeps its state in explicit coordinates. As in PAL, every geometric quantity comes from a deterministic tool; unlike PAL's interpreter, which solves the whole problem, each tool is restricted to a single timestep: it is a function of one stop's detections, the previous belief, and a few harness counters, for example composing one odometry increment, converting one range and bearing into a map point, or fixing the pose from the beacons seen at that stop. None optimizes over the observation history. The system prompt prescribes a per-step procedure (predict, associate, correct the pose, update the map, check loop closure, choose a move), and the model carries it out by calling the tools and acting on their results, so trusting, matching, and accepting remain steps that the model must execute correctly. Because that procedure can also be run as ordinary code, a rule-program baseline is proposed to measure what executing it with an LLM costs in accuracy, time, and money; what the LLM adds beyond the procedure, such as handling free-text requests and refusing or asking, would be scored by the request suite. Each decision is a fresh, stateless conversation that carries only the mission, any operator request, the model's previous report, and the new observation. Every step is thus posed as an isolated decision, which Jiang *et al.* measured to be more reliable than the same decision made inside a long plan. Whether that is enough is open: in their study the collapse returned even with per-junction queries, and the maze studies used symbolic grids with perfect observations, whereas BeaconMind poses a physical estimation problem with noisy odometry and noisy range and bearing measurements.

D. Fiducial Landmark SLAM and Evaluation Practice

Fiducial SLAM supplies both the landmarks and the reference solution. AprilTag is a visual fiducial system that gives a tag's identity and its full 6-DOF pose relative to the camera from a single image [23]. TagSLAM builds a factor graph over tags, cameras, and bodies on the GTSAM factor-graph library and observes that, with a small set of artificial landmarks, "loop closure is guaranteed once a tag is successfully decoded" [24]; iSAM2 solves such graphs incrementally through the Bayes tree [25]. For evaluation, Sturm *et al.* formalize the absolute trajectory error (ATE) and the relative pose error (RPE) as the trajectory metrics of their SLAM benchmark [26], and Anderson *et al.* recommend that a navigation agent signal completion with an explicit "DONE" action and be scored by success weighted by path length (SPL) [27].

BeaconMind uses this body of work in two ways. First, fiducial SLAM supplies the yardstick: a GTSAM/iSAM2 planar factor graph is the best-case reference and dead reckoning (odometry alone) is the floor, and the LLM's ATE, RPE, and beacon-map error will be reported against both on identical replayed logs from simulation and from the real car. In replay the moves are those recorded in the log and the model only estimates; move choice is evaluated separately in closed loop, where navigation with the finished map will be scored by ground-truth success at "DONE" and by SPL. Second, BeaconMind removes the assumption that makes data association in fiducial SLAM trivial. TagSLAM requires unique identifiers, so decoding a tag settles data association and, with it, loop closure. BeaconMind's benchmark adds a condition in which identifiers are duplicated on purpose, so that association and, when a revisit occurs, loop closure require reasoning over geometry and the map built so far. For that condition it is proposed that the reference receive the true association, which makes it an upper bound on achievable accuracy, not a like-for-like competitor. The systems in Table I either face no landmark association or solve it in classical code. SEO-SLAM is a partial exception: its measurement-to-landmark association is a linear assignment, but its language-model agents afterwards merge duplicate landmarks and remove erroneous ones. Whether an LLM can make these decisions with useful precision, and at what latency and cost, is the question BeaconMind is built to answer.

III. System Overview

Reserved for a later draft.

IV. Experiments

Reserved for a later draft.

V. Results and Evaluation

Reserved for a later draft.

VI. Conclusion

Reserved for a later draft.

GENERATIVE AI DISCLOSURE

Claude Fable 5.1 (Anthropic), used through Claude Code, assisted with this draft: it searched for and retrieved the cited papers, checked each summarized claim against the paper's text, drafted the wording of Section II and Table I, prepared the BIBTEX entries and the LATEX source, and ran an adversarial review of the draft. Many of the sources were first identified in the project's earlier AI-assisted literature searches (Claude Opus 5.5 in Claude Code and in Cursor, with citations re-checked by Grok 4.6). The project's design documents, on which the descriptions of BeaconMind rest, were drafted, and many of their detailed design choices made, by Claude Opus 5.5 under the author's delegation; the author's review of some of those choices is still in progress. The author chose the project direction, reviews all AI-assisted content against the sources before submission, and is responsible for its accuracy and originality.

REFERENCES

[1] W. Huang, P. Abbeel, D. Pathak, and I. Mordatch, "Language models as zero-shot planners: Extracting actionable knowledge for embodied agents," in *Proceedings of the 39th International Conference on Machine Learning*, vol. 162. PMLR, 2022, pp. 9118–9147.

[2] M. Ahn *et al.*, "Do as I can, not as I say: Grounding language in robotic affordances," *arXiv preprint arXiv:2204.01691*, 2022, also in Proc. 6th Conf. Robot Learning (CoRL), PMLR vol. 205, pp. 287–318, 2023.

[3] W. Huang *et al.*, "Inner Monologue: Embodied reasoning through planning with language models," in *Proceedings of the 6th Conference on Robot Learning*, vol. 205. PMLR, 2023, pp. 1769–1782.

[4] S. Yao *et al.*, "ReAct: Synergizing reasoning and acting in language models," in *International Conference on Learning Representations (ICLR)*, 2023.

[5] S. Kambhampati *et al.*, "Position: LLMs can't plan, but can help planning in LLM-Modulo frameworks," in *Proceedings of the 41st International Conference on Machine Learning*, vol. 235. PMLR, 2024, pp. 22 895–22 907.

[6] R. Royce *et al.*, "Enabling novel mission operations and interactions with ROSA: The Robot Operating System Agent," in *2025 IEEE Aerospace Conference*, 2025, pp. 1–16.

[7] S. Salimpour *et al.*, "Towards embodied agentic AI: Review and classification of LLM- and VLM-driven robot autonomy and interaction," *arXiv preprint arXiv:2508.05294*, 2025.

[8] M. Li *et al.*, "Embodied Agent Interface: Benchmarking LLMs for embodied decision making," in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, vol. 37, 2024, pp. 100 428–100 534.

[9] S. Yin *et al.*, "SafeAgentBench: A benchmark for safe task planning of embodied LLM agents," *arXiv preprint arXiv:2412.13178v5*, 2025.

[10] K. Rana, J. Haviland, S. Garg, J. Abou-Chakra, I. Reid, and N. Sünderhauf, "SayPlan: Grounding large language models using 3D scene graphs for scalable robot task planning," in *Proceedings of the 7th Conference on Robot Learning*, vol. 229. PMLR, 2023, pp. 23–72.

[11] N. Yokoyama, S. Ha, D. Batra, J. Wang, and B. Bucher, "VLFM: Vision-language frontier maps for zero-shot semantic navigation," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*, 2024, pp. 42–48.

[12] B. Yu, H. Kasaei, and M. Cao, "L3MVN: Leveraging large language models for visual target navigation," in *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2023, pp. 3554–3560.

[13] J. Lee, J. Park, and W. Kim, "Spatial and semantic reasoning for LLM-driven robot navigation via MCP," *arXiv preprint arXiv:2609.27340*, 2026.

[14] A. H. M. Syarubany, F. Z. Rahmani, and T. Widianto, "LLM-based agentic exploration for robot navigation & manipulation with skill orchestration," *arXiv preprint arXiv:2601.00555*, 2026.

[15] J. Hong, R. Choi, and J. J. Leonard, "Semantic enhancement for object SLAM with heterogeneous multimodal large language model agents," in *2025 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2025, pp. 9036–9043.

[16] S. Sandfuchs, M. Melchert, and J. Frochte, "LLMs for text-based exploration and navigation under partial observability," *arXiv preprint arXiv:2604.09604*, 2026.

[17] Z. Lyu, J. Zhang, M. Lu, Y. Li, and C. Feng, "Tell me where you are: Multimodal LLMs meet place recognition," *arXiv preprint arXiv:2406.17520*, 2024.

[18] A. Sridhar, D. Shah, C. Glossop, and S. Levine, "NoMaD: Goal masked diffusion policies for navigation and exploration," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*, 2024, pp. 63–70.

[19] Y. Jiang, P. Luo, and L. Meng, "Lost in aggregation: A multi-scale diagnostic benchmark for LLM spatial navigation," *arXiv preprint arXiv:2606.22219*, 2026.

[20] W. Li, Y. Zhu, R. Das, and P. Dube, "Do LLMs build spatial world models? Evidence from grid-world maze tasks," *arXiv preprint arXiv:2604.10690*, 2026, ICLR 2026 Workshop on World Models.

[21] K. Deng, Y. Qiu, Y. Kasten, S. B. Cohen, and Y. Ziser, "Lost in space? Vision-language models struggle with relative camera pose estimation," *arXiv preprint arXiv:2601.22228v2*, 2026.

[22] L. Gao *et al.*, "PAL: Program-aided language models," in *Proceedings of the 40th International Conference on Machine Learning*, vol. 202. PMLR, 2023, pp. 10 764–10 799.

[23] E. Olson, "AprilTag: A robust and flexible visual fiducial system," in *2011 IEEE International Conference on Robotics and Automation*, 2011, pp. 3400–3407.

[24] B. Pfrommer and K. Daniilidis, "TagSLAM: Robust SLAM with fiducial markers," *arXiv preprint arXiv:1910.00679*, 2019.

[25] M. Kaess, H. Johannsson, R. Roberts, V. Ila, J. J. Leonard, and F. Dellaert, "iSAM2: Incremental smoothing and mapping using the Bayes tree," *The International Journal of Robotics Research*, vol. 31, no. 2, pp. 216–235, 2012.

[26] J. Sturm, N. Engelhard, F. Endres, W. Burgard, and D. Cremers, "A benchmark for the evaluation of RGB-D SLAM systems," in *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2012, pp. 573–580.

[27] P. Anderson *et al.*, "On evaluation of embodied navigation agents," *arXiv preprint arXiv:1807.06757*, 2018.